



University of
Inland Norway

ØKA2015 Applied Data Science

Autumn 2026 | 7.5 study points | Lillehammer

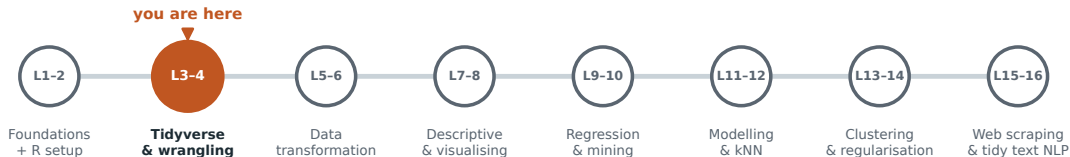
Lecture 3

Introduction to the Tidyverse

Touseef Hameed | `touseef.hameed@inn.no`

Wednesday 16 September 2026 | 13:15–15:00 | Room G102

Where we are in the course



Last time (Lecture 2, 27 August)

The six-step data science process, tidy data, what R and RStudio are, and you installed both. The last command you typed was `1 + 1`.

Today

You learn to *instruct* R: objects, vectors, functions. Then you install the tidyverse and read your first real dataset.

Materials for this lecture

Everything below is in one place online, and stays there for the whole course. Open it now and keep the tab open.

Slides | this document, as a PDF

R script | `lecture03_tidyverse.R`: every command from today, with a comment explaining each line and each operator. Run it line by line in RStudio.

R Markdown tutorial | `lecture03_tidyverse.Rmd`, and its knitted PDF: the same content as a document with text and code together, which is the form your assignments take

Practice exercise | six short tasks, not graded, the warm-up for Assignment 1

Data | `students.csv`, six students, five columns

The GitHub page is additional material. The lab today is on your own laptop, because that is where Assignment 1 is written.

Where it is

GitHub, the course repository:

<https://github.com/touseefhameed/oka2015-applied-data-science>

Google Colab, the script running in your browser with nothing to install:

[open the notebook](#)

Reading

Wickham et al., *R for Data Science* (2e), free online: the Introduction (*Prerequisites*), Ch. 2 *Workflow: basics*, Ch. 7 *Data import*.

<https://r4ds.hadley.nz>

Today's plan

1 Instructing R

calculator, objects, vectors, functions, comments, names, precision

2 Packages

install once, load every session

3 The tidyverse

nine packages, one per task

4 Tibbles and reading data

`read_csv()`, three routes in, a first look at the data, other file types

5 Real data is rarely clean

missing values and column types, a preview of Friday

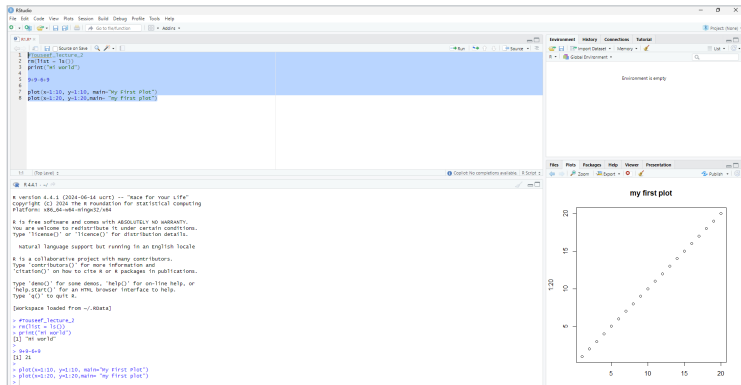
6 Data Lab: install the tidyverse, load a dataset, save a script

By the end you will be able to

- ✓ store values in objects and vectors
- ✓ call a function with arguments
- ✓ install and load a package
- ✓ read a CSV file into R
- ✓ report how many rows and columns it has, and what each column contains

That last line is Assignment 1,
Problems 1 to 4.

Where you type: a one-minute recap of RStudio



Console (bottom left)

Type a command, press **Enter**, R answers. Nothing is saved.

Source (top left)

Write commands you want to keep.

Ctrl + Enter runs one line.

Environment (top right)

Every object you have created.
Empty at the moment.

Files, Plots, Packages, Help

Your folders, your graphs, and the documentation.

1. R as a calculator

Type in the **Console** and press Enter:

```
1 + 1  
1 / 200 * 30  
(59 + 73 + 2) / 3  
sin(pi / 2)
```

```
[1] 2  
[1] 0.15  
[1] 44.66667  
[1] 1
```

`[1]` | R indicates that this line of output starts at element number 1 of the result. It becomes meaningful once results have several elements.

`pi` | a built-in constant. `sqrt()`, `log()` and `round()` are built-in functions.

Parentheses

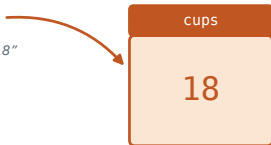
Work as in mathematics. `(59 + 73 + 2) / 3` is an average. Without the parentheses, only the 2 would be divided.

R for Data Science (2e), Ch. 2 Workflow: basics, <https://r4ds.hadley.nz/workflow-basics>

1. Objects: the assignment operator <-

```
cups <- 18
```

read it as: "cups gets 18"



an object: a named box in R's memory

```
cups * 2
```

```
[1] 36
```

*use the name, and R
looks inside the box*

```
cups <- 18 # nothing is printed  
cups # the name alone displays it  
price <- 52 # one latte, in NOK  
cups * price
```

Object | a named location in R's memory that holds a value. Everything you store is an object.

<- | the **assignment operator**. Every assignment has the form `object_name <- value`; read it as "object name gets value". Shortcut in RStudio: **Alt + minus**.

The coffee bar from Lecture 2 is the running example for the whole lecture.

1. Vectors: `c()` combines values

```
cups <- c(18, 22, 27)
```

c() combines values into one object, a vector
cups



```
cups * 52
```

one instruction, applied to every element
revenue (NOK)



No loop needed. Most of R works like this.

```
cups <- c(18, 22, 27) # Mon, Tue, Wed  
cups * price  
sum(cups)  
mean(cups)
```

```
[1] 936 1144 1404  
[1] 67  
[1] 22.33333
```

Vector | several values of the same kind, held in one object. A column of a table is a vector.

Key idea

R applies one instruction to **every element at once**. This is called vectorised computation, and it is why R code rarely needs a loop.

1. Functions and arguments

the function's name
(what to do)

↓

read_csv

("students.csv" , na = "N/A")

↑

first argument
(what to do it to)

a named argument
(how to do it)

Arguments go inside the parentheses, separated by commas. Text always sits in quotation marks.

```
seq(from = 1, to = 10) # named arguments
seq(1, 10) # same, by position
round(mean(cups), 1) # a function inside a
                    function
?mean # opens the help page
```

Function | a named command that performs one task. General form: `name(argument1 = value1, argument2 = value2)`.

Argument | what you pass to the function. The first few may be given by position; later ones are clearer by name.

?name | the documentation for any function, in the Help pane.

```
[1] 1 2 3 4 5 6 7 8 9 10
[1] 22.3
```

1. Comments and object names

Comments

```
# create a vector of cups sold  
cups <- c(18, 22, 27)  
cups * 2 # double every element
```

| R ignores everything after # on that line. A comment is text for the human reader, explaining *why* the code does what it does.

Names

```
i_use_snake_case  
otherPeopleUseCamelCase  
some.people.use.periods
```

Rule | a name must start with a letter, and may contain only letters, numbers, _ and .

Convention | choose descriptive names, and separate words consistently. This course uses **snake_case**: lower-case words joined by underscores, as in `cups_monday`.

In your assignments

A comment above every command is required. It is how the reader, and you in three weeks, understand what was intended.

R for Data Science (2e), Ch. 2, sections Comments and What's in a name?

1. Precision: R does exactly what you type

1. R is **case-sensitive**

```
cups  
Cups
```

```
[1] 18 22 27  
Error: object 'Cups' not found
```

2. Text requires quotation marks; numbers do not

```
shop <- "Kaffebar" # text  
year <- 2026 # a number  
town <- Lillehammer # error, see right
```

```
Error: object 'Lillehammer' not found
```

3. Quotation marks and parentheses come in pairs

```
x <- "hello
```

```
+
```

The + prompt | R is waiting for the rest of the command, usually a closing " or). Either complete it, or press **Esc** and start again.

Reading an error message

The message is informative: it names the object R could not find. In most cases the cause is a typing error, a difference in capitalisation, or a missing quotation mark.

R for Data Science (2e), Ch. 2: "R will do the tedious computations for you, but in exchange, you must be completely precise in your instructions."

1. RStudio helps you type

Tab | completion. Type the first letters of a name and press Tab; RStudio lists the possible completions and, for a function, shows its arguments.

↑ | recalls the previous command, so you can correct it instead of retyping. **Ctrl + ↑** lists earlier commands that start with what you typed.

F1 | with the cursor on a function name, opens its help page.

Pairs | typing (or " inserts the closing partner automatically.

R for Data Science (2e), Ch. 2, section Calling functions

Why this matters on day one

Most early errors are typing errors. Completion removes most of them, and the history arrow means a mistake costs one keystroke, not a retype.

Try it now, one minute

In the Console type `se` and press **Tab**. Choose `seq`. Press **Tab** again inside the parentheses to see its arguments.

1. The Environment: what R currently holds

```
ls() # list every object
```

```
[1] "cups" "price" "shop" "year"
```

```
rm(shop) # remove one object  
rm(list = ls()) # remove all objects  
ls()
```

```
character(0)
```

Environment | the set of objects created in the current session. The Environment pane shows the same list as `ls()`.

Objects are not files

Objects exist in memory only. When RStudio closes, they are gone. The commands that created them belong in a **script**; running the script recreates the objects.

Your turn, two minutes

Store the three prices 52, 32 and 30 in `prices`. Multiply by `cups`. Interpret the result.

2. Packages: reusing code written by others

A package is a bundle of ready-made functions someone else wrote for you.



```
install.packages("tidyverse") # once per  
computer  
library(tidyverse) # every session
```

Package | a collection of functions, data and documentation that extends R. Packages are the fundamental unit of reproducible R code. Over 20,000 exist on **CRAN** (Comprehensive R Archive Network).

Note the quotation marks

`install.packages("tidyverse")` **with** quotes;
`library(tidyverse)` **without**. This inconsistency is a common source of early errors.

2. A common error, and its remedy

```
library(janitor)
```

```
Error in library(janitor) :  
  there is no package called 'janitor'
```

The message is literal: the package is not installed on *this* computer. Install it, then load it.

```
install.packages("janitor")  
library(janitor)
```

`package::function()` | names the package explicitly when two packages provide a function with the same name: `dplyr::filter()` versus `stats::filter()`. You will meet this on Friday.

Where packages come from

`install.packages()` downloads from **CRAN**, the same site R itself came from, and therefore needs an internet connection. `library()` does not.

3. What the tidyverse is

tidyverse

A **collection of packages** that share a common philosophy of data and of R programming, and are designed to work together. Most of the packages used in this course belong to it.

One installation provides all of them:

Core | the nine packages loaded by `library(tidyverse)`

Additional | installed at the same time but loaded individually when needed: `readxl` for Excel files, `jsonlite`, `DBI` and others

“Tidy” has the meaning from Lecture 2: columns are variables, rows are observations.

<https://www.tidyverse.org>

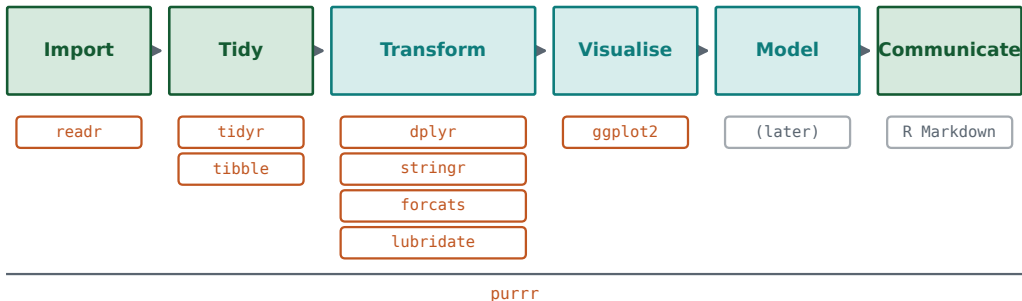
Components



3. Nine packages, one per task

The tidyverse, laid on top of the process flow from Lecture 2

one package per job, all designed to work together



The same process diagram as Lecture 2. Each step now has a named tool. Today: **readr** and **tibble**. Friday: **dplyr**.

3. The nine core packages

Package	Purpose	Introduced in
readr	reading rectangular data files (CSV, TSV) into R	today
tibble	the modern data frame	today
dplyr	data manipulation: filter, select, mutate, summarise	Lecture 4
tidyr	reshaping data into tidy form	Lectures 5 to 6
ggplot2	graphics, based on the grammar of graphics	Lectures 7 to 8
stringr	working with text (strings)	Lecture 15
forcats	categorical variables (factors)	later
lubridate	dates and times	later
purrr	applying a function repeatedly (functional programming)	later



Logos: <https://www.tidyverse.org>. This table is for reference; only readr and tibble are needed today.

3. What library(tidyverse) prints, and what it means

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----
  tidyverse 2.0.0 --
v dplyr 1.1.4 v readr 2.1.5
v forcats 1.0.0 v stringr 1.5.1
v ggplot2 3.5.1 v tibble 3.2.1
v lubridate 1.9.3 v tidyr 1.3.1
v purrr 1.0.2
-- Conflicts -----
  tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag() masks stats::lag()
```

The check marks | nine packages loaded. Version numbers on your laptop may differ; that is expected.

Conflicts | two packages define a `filter()`; the tidyverse version takes precedence. This is **information, not an error**.

`tidyverse_update()` | reports whether newer versions are available. The packages change fairly often.

If the text is red and says Error

That is different. Read it, and raise your hand.

4. The tibble: the tidyverse's data table

```
students
```

```
# A tibble: 6 x 5  
  'Student ID' 'Full Name' favourite.food ...  
    <dbl> <chr> <chr>  
1 1 Sunil Huffmann Strawberry yoghurt  
2 2 Barclay Lynn French fries  
3 3 Jayendra Lyne N/A  
4 4 Leon Rossini Anchovies  
5 5 Chidiegwu Dunkel Pizza  
6 6 Guvenc Attila Ice cream
```

Tibble | a table of data in which columns are variables and rows are observations. It is the tidyverse's refinement of R's original *data frame*.

What it shows you

- its **dimensions** first: 6 x 5, rows by columns
- the **type** of every column, under its name
- only the first 10 rows, so a large dataset remains readable

<dbl> | a number ("double")

<chr> | text ("character")

4. The file we are about to read

`students.csv`, as stored on disk

```
Student ID,Full Name,favourite.food,  
mealPlan,AGE  
1,Sunil Huffmann,Strawberry yoghurt,Lunch  
only,4  
2,Barclay Lynn,French fries,Lunch only,5  
3,Jayendra Lyne,N/A,Breakfast and lunch,7  
4,Leon Rossini,Anchovies,Lunch only,  
5,Chidiegwu Dunkel,Pizza,Breakfast and  
lunch,five  
6,Guvenc Attila,Ice cream,Lunch only,6
```

CSV (Comma-Separated Values) | from Lecture 2:
a plain text file in which a comma separates the
columns (the *delimiter*) and a new line separates
the rows.

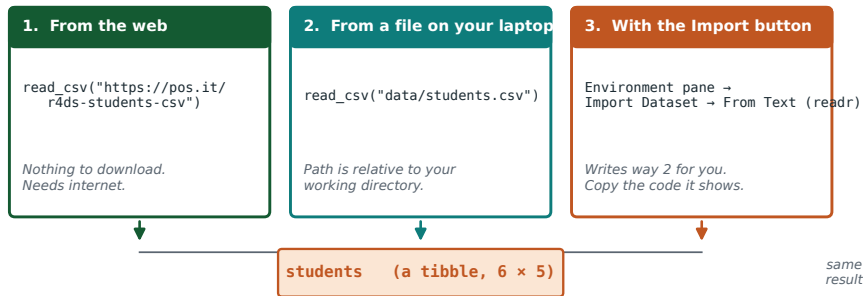
Header row | the first line. It supplies the column
names.

Observe

Row 3 contains the text `N/A`. Row 4 has nothing
after the last comma. Row 5 gives an age as the
word `five`. Real data files commonly contain such
irregularities; we return to all three.

From *R for Data Science*, Ch. 7. Six students, five
columns.

4. read_csv(): three routes to the same tibble



```
students <- read_csv("https://pos.it/r4ds-students-csv") # route 1, used in today's lab
```

Whichever route you take, the line that belongs in your script is a `read_csv()` call. The first argument, the path, is the most important one.

4. Route 2 needs one idea: the working directory

```
getwd()
```

```
[1] "C:/Users/you/Documents"
```

```
read_csv("data/students.csv")
```

means: look inside `C:/Users/you/Documents/data/` for `students.csv`.

```
setwd("C:/Users/you/OneDrive/oka2015")
```

Working directory | the folder R reads from and writes to unless told otherwise. `getwd()` reports it; `setwd()` changes it.

Relative path | `data/students.csv`: a path that starts from the working directory. Short, and valid on any computer with the same folder layout.

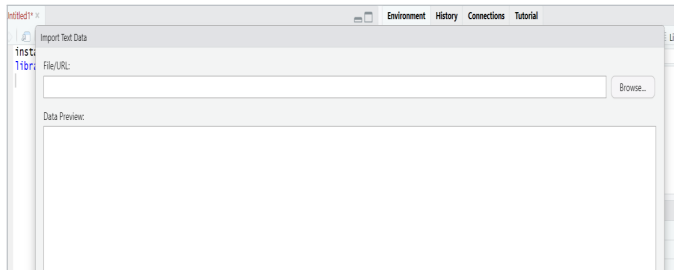
Windows users

Write paths with forward slashes `/`. R interprets a backslash as the start of a special character.

The recommended way

File → New Project in RStudio. The working directory is then always the project folder and `setwd()` is never needed.

4. Route 3: the Import button writes the code for you



Environment pane → **Import Dataset**
→ **From Text (readr)** → Browse to the file.

The dialog previews the data and shows, at the bottom right, the exact `read_csv()` command it is about to run.

Copy that command into your script

The dialog is convenient once. The command in a script is what makes the import reproducible, and it is what you submit.

4. A first look: always inspect before you analyse

```
head(students) # first six rows
nrow(students) # rows: observations
ncol(students) # columns: variables
names(students) # column names
glimpse(students) # every column, one per
  line
summary(students) # basic statistics
View(students) # spreadsheet view, capital V
```

```
[1] 6
[1] 5
[1] "Student ID" "Full Name" "favourite.food"
"
[4] "mealPlan" "AGE"
```

`glimpse()` | the tidyverse overview: rows, columns, and every variable with its type on one line. Use it first.

This is Assignment 1, Problems 1 to 4

Load the file, display the first rows, count observations and variables, summarise. You are now able to do it.

```
num_rows <- nrow(students)
num_cols <- ncol(students)
cat("Rows:", num_rows, " Cols:",
    num_cols, "\n")
```

Rows: 6 Cols: 5

4. Reading the output of `glimpse()`

```
glimpse(students)
```

```
Rows: 6  
Columns: 5  
$ 'Student ID' <dbl> 1, 2, 3, 4, 5, 6  
$ 'Full Name' <chr> "Sunil Huffmann", "Barclay Lynn", "Jayendra Lyne", "Leon Ross...  
$ favourite.food <chr> "Strawberry yoghurt", "French fries", "N/A", "Anchovies", "Pi...  
$ mealPlan <chr> "Lunch only", "Lunch only", "Breakfast and lunch", "Lunch onl...  
$ AGE <chr> "4", "5", "7", NA, "five", "6"
```

`$` | the operator that selects one column from a table:
`students$AGE`. You will use it constantly.

Backticks `'Student ID'` | a name containing a space is *non-syntactic* and must be written between backticks. On Friday we rename such columns.

Two observations

`AGE` is `<chr>`, text, although it should be numeric.
`favourite.food` contains `"N/A"` in quotation marks: text, not a missing value. Both are incorrect, and both are typical.

4. Several files at once, and writing a file

Data split over several files, one per month, can be read in one call:

```
files <- c("data/01-sales.csv",  
          "data/02-sales.csv",  
          "data/03-sales.csv")  
sales <- read_csv(files, id = "file")
```

```
# A tibble: 19 x 6  
  file month year brand item n  
1 data/01-sales.csv January 2019 1 1234 3  
2 data/01-sales.csv January 2019 1 8721 9
```

`id = "file"` | adds a column recording which file each row came from, so observations can be traced to their source.

R for Data Science (2e), Ch. 7, sections Reading data from multiple files and Writing to a file

The reverse operation:

```
write_csv(students, "students-2.csv")
```

`write_csv()` | saves a tibble as a CSV file. Arguments: the object, then the path.

Note

A CSV stores values only, not column types. Corrected types must be corrected again after reading the file back.

4. Other file types: the same idea, a different function

Once `read_csv()` is familiar, the rest of `readr` is a matter of choosing the function that matches the file.

Function	Reads
<code>read_csv2()</code>	semicolon-separated files. Common in Norway , where the comma is the decimal marker, so Excel saves CSV with ;
<code>read_tsv()</code>	tab-separated files
<code>read_delim()</code>	any delimiter; it guesses if you do not specify one
<code>read_fwf()</code>	fixed-width files, columns defined by position
<code>read_table()</code>	columns separated by white space
<code>read_log()</code>	web server log files
<code>readxl::read_excel()</code>	Excel workbooks (.xlsx); <code>readxl</code> is installed with the tidyverse but loaded separately

If a Norwegian Excel export reads as one column, the delimiter is ; and you need `read_csv2()`.

R for Data Science (2e), Ch. 7, section Other file types

5. “N/A” is not NA

To R, `N/A` is three characters of text, not a missing value. State what missing looks like in *this* file:

```
students <- read_csv("https://pos.it/r4ds-
  students-csv",
                    na = c("N/A", ""))
students$favourite.food
sum(is.na(students))
colSums(is.na(students))
```

```
[1] "Strawberry yoghurt" "French fries" NA
...
[1] 2
Student ID Full Name favourite.food
      0 0 1
mealPlan AGE
      0 1
```

R for Data Science (2e), Ch. 7, section *Practical advice*

NA | R's representation of a **missing** value: not zero, not text, not an error. Not available.

na = c(...) | a named argument: “treat these strings as missing”. The empty string `""` is included by default; `"N/A"` is not.

is.na() | **TRUE** where a value is missing. `sum()` counts them, because **TRUE** counts as 1.

5. “five” is not 5

One age was entered as a word, so `read_csv()` classified the *whole* column as text. Correct the word, then convert the column:

```
students$AGE <- gsub("five", "5",  
  students$AGE)  
students$AGE <- as.numeric(students$AGE)  
students$AGE  
mean(students$AGE, na.rm = TRUE)
```

```
[1] 4 5 7 NA 5 6  
[1] 5.4
```

`gsub()` | global substitution: replace every occurrence of “five” with “5” in that column.

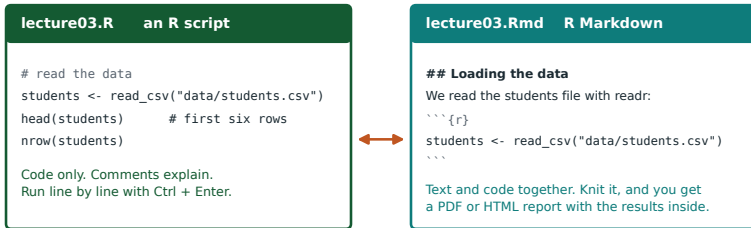
`as.numeric()` | converts text to numbers. Any value that is not a number becomes `NA`, with a warning.

Why this matters

The mean of a text column cannot be computed. One stray word blocks every calculation on the column, so checking column types is the first step with any new dataset.

Friday: the same corrections, done the tidyverse way with `mutate()`.

6. Two file types you will use throughout the course



R script .R | code and comments only. For working and for debugging. **File → New File → R Script.**

R Markdown .Rmd | text and code together, knitted into a PDF or a web page. For reporting and for submission. **File → New File → R Markdown.**

Assignment 1 requires both: a script, and an R Markdown document.

6. Course materials online

Everything from this lecture is online, and remains there for the whole course:

GitHub | <https://github.com/touseefhameed/oka2015-applied-data-science>

the slides, the R script, the R Markdown document and its PDF, the data, and a practice exercise, for every lecture

Google Colab | [open the notebook](#)

the same script, running in your browser. No installation. Use it if your own installation is not yet working.

Colab in three steps

Open the link → sign in with any Google account → press the play button on the first cell. The notebook is already set to R.

Additional material, not a replacement

The lab today is on **your own laptop**, because that is where Assignment 1 is written. Colab is the fallback.

Two links to keep

- ✓ the GitHub repository
- ✓ <https://r4ds.hadley.nz>, the free textbook

Check your understanding

1. Why does this code not work?

```
my_variable <- 10  
my_variabel
```

2. Which two lines install the tidyverse and load it for use? What is wrong with the others?

```
install.packages("tidyverse")  
install_packages("tidyverse")  
library(tidyverse)  
tidyverse = install
```

3. When would you use an R Markdown file rather than a script?

Check your understanding

1. Why does this code not work?

```
my_variable <- 10  
my_variabel
```

2. Which two lines install the tidyverse and load it for use? What is wrong with the others?

```
install.packages("tidyverse")  
install_packages("tidyverse")  
library(tidyverse)  
tidyverse = install
```

3. When would you use an R Markdown file rather than a script?

Answers

1. The object was created as `my_variable` and requested as `my_variabel`. R matches names exactly, so the second is unknown.

2. The first and the third. `install_packages` does not exist (the dot matters), and the last line assigns the object `install` to a name, which is not a command.

3. An `.Rmd` when the result is a document that combines explanation, code and output, as in an assignment. A `.R` script for working, testing and reusable code.

Data Lab

Install the tidyverse, read your first dataset, save a script

Laptops open. We work through this together.

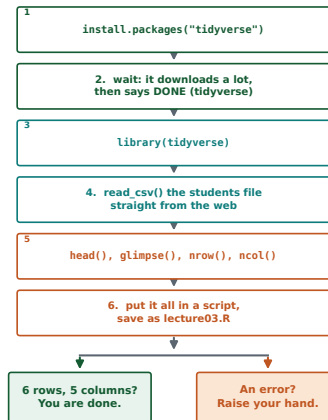
Data Lab: the goal

You leave this room with

- ✓ the tidyverse installed and loaded
- ✓ `students.csv` read into R from the web
- ✓ its rows, columns, names and types on your screen
- ✓ all of it saved in a script called `lecture03.R`

If you are stuck

Raise your hand. If your installation is not working, open the Colab notebook from the materials slide and follow along there; we repair the installation after class.



This is the whole of Assignment 1, Problems 1 to 4.

Lab step 1: install the tidyverse

In the **Console**, type and press Enter:

```
install.packages("tidyverse")
```

```
Installing package into 'C:/Users/you/...'
trying URL 'https://cran.rstudio.com/...'
Content type 'application/zip' length 431857
      bytes
downloaded 421 KB
...
package 'tidyverse' successfully unpacked
```

About 40 packages are downloaded. On the campus network this takes **two to four minutes**. Wait until the **>** prompt returns.

Common installation problems

It asks “do you want to install from sources?” Type **no** and press Enter.

A window asks you to choose a **CRAN mirror**. Any is fine; Norway or 0-Cloud.

“cannot open URL” means no internet connection. Check Wi-Fi and retry.

Warning: package ... is in use. Session → Restart R, then rerun.

Lab step 2: load it

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----
  tidyverse 2.0.0 --
v dplyr 1.1.4 v readr 2.1.5
v forcats 1.0.0 v stringr 1.5.1
v ggplot2 3.5.1 v tibble 3.2.1
v lubridate 1.9.3 v tidyr 1.3.1
v purrr 1.0.2
-- Conflicts -----
  tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag() masks stats::lag()
```

Nine check marks and a conflicts note indicate a **successful load**.

No quotation marks this time

`library(tidyverse)`, not `library("tidyverse")`. Both happen to work; learn the form without.

Every session

Tomorrow, when you open RStudio, the packages are on disk but *not loaded*. `library(tidyverse)` is always the first line of your script.

Lab step 3: read the file and inspect it

```
students <- read_csv("https://pos.it/r4ds-  
  students-csv")  
students  
head(students)  
nrow(students)  
ncol(students)  
glimpse(students)
```

```
Rows: 6 Columns: 5  
-- Column specification -----  
Delimiter: ","  
chr (4): Full Name, favourite.food, mealPlan  
  , AGE  
dbl (1): Student ID
```

That block is `read_csv()` reporting what it found: five columns, four of text and one numeric. It is not an error.

Check yourself

- ✓ `nrow` reports 6, `ncol` reports 5
- ✓ the Environment pane shows `students`, 6 *obs. of 5 variables*
- ✓ clicking it there opens a spreadsheet view

No internet?

Ask for the file on a USB stick, place it in a `data` folder, and use `read_csv("data/students.csv")`.

Lab step 4: put it in a script and save it

File → New File → R Script. Type this, with a comment on every line:

```
# Lecture 3, 16 September 2026
library(tidyverse) # load

# read the students file from the web
students <- read_csv("https://pos.it/r4ds-
  students-csv")

head(students) # first six rows
nrow(students) # rows
ncol(students) # columns
glimpse(students) # types
```

Save it: **Ctrl + S**, named `lecture03.R`.

Ctrl + Enter | runs the line under the cursor and moves to the next. Run the script from top to bottom this way.

Ctrl + Shift + Enter | runs the whole file.

The test that matters

Close RStudio. Open it again. Open `lecture03.R` and run it from the top. If it runs with nothing else typed, the analysis is reproducible, which is the standard for every assignment.

Lab: when something goes wrong

What you see	What to do
could not find function "read_csv"	<code>library(tidyverse)</code> has not been run in this session. Run it.
there is no package called 'tidyverse'	The installation did not complete. Run <code>install.packages("tidyverse")</code> again.
Error: object 'students' not found	The <code>read_csv</code> line has not been run, or the name is typed differently.
cannot open file 'data/students.csv'	Wrong working directory. <code>getwd()</code> to see where you are, or use the web address.
Console shows + and does not respond	An unfinished command, usually a missing quotation mark or bracket. Press Esc .
Repeated red errors, nothing works	Session → Restart R, then <code>library(tidyverse)</code> again.

Read the message. It states precisely what R could not do.

What we covered today

Concepts

- ✓ objects and the assignment operator `<-`
- ✓ vectors and vectorised computation
- ✓ functions, arguments, comments, names
- ✓ packages: install once, load every session
- ✓ the tidyverse: nine packages, one per task
- ✓ tibbles, `NA`, and column types

Tools

- ✓ `c()`, `sum()`, `mean()`, `seq()`, `ls()`, `rm()`
- ✓ `install.packages()`, `library()`
- ✓ `read_csv()` by three routes; `write_csv()`
- ✓ `head()`, `nrow()`, `ncol()`, `glimpse()`, `summary()`
- ✓ `getwd()`, `setwd()`, Tab completion
- ✓ a saved, rerunnable script

If `glimpse(students)` shows 6 rows and 5 columns on your laptop, today's objective is met.

Assignment 1 is published today

Assignment 1: readr and dplyr

Posted on Canvas this afternoon. Due **Wednesday 30 September, 23:59**.

Eight problems. Six require short pieces of R code, two require short written answers. You submit a script *and* an R Markdown document.

Problems 1 to 4 | `read_csv`, `head`, `nrow`, `ncol`, `summary`. Covered **today**.

Problems 5 to 6 | `select()`, `mutate()`. Covered Friday.

Problems 7 to 8 | what dplyr is for, what wrangling is. Lecture 2 and Friday.

Start early

Half of the assignment is what you did in the lab, applied to a different file. Completing Problems 1 to 4 before Friday means Friday's lecture answers a question you already have.

Before Friday

- ✓ complete the lab if you did not finish
- ✓ do the practice exercise on GitHub
- ✓ read *R for Data Science* Ch. 3, the first two sections

Next: Lecture 4, Friday 18 September, Room S217

Data transformation with dplyr

The verbs that perform most of the work in this course:

- `filter()` keep the rows you want
- `select()` keep the columns you want
- `mutate()` create a new column from existing ones
- `arrange()` sort
- `group_by()` + `summarise()` totals and averages per group

and the pipe `|>`, which chains them into one readable sequence.

The thread

Today you brought data *into* R and inspected it.

Friday you begin *transforming* it, which is the wrangling that takes 50 to 80% of the work, and Problems 5 and 6.

Bring the laptop with the tidyverse installed.
Friday assumes it.

Sources used in this lecture

Course reading

- Wickham, H., Çetinkaya-Rundel, M. & Grolemund, G. (2023). *R for Data Science* (2nd ed.). O'Reilly. Introduction, *Prerequisites* | Ch. 2 *Workflow: basics* | Ch. 7 *Data import*. Free online: <https://r4ds.hadley.nz>
- Wright, C., Ellis, S., Hicks, S. & Peng, R. (2021). *Tidyverse Skills for Data Science in R*, Ch. 3. <https://jhudatascience.org/tidyversecourse/>

Packages

- The tidyverse: <https://www.tidyverse.org>
- readr: <https://readr.tidyverse.org> tibble: <https://tibble.tidyverse.org>
- Posit cheat sheets, one page per package: <https://posit.co/resources/cheatsheets/>

Course material

- GitHub: <https://github.com/touseefhameed/oka2015-applied-data-science>
- Google Colab notebook: https://colab.research.google.com/github/touseefhameed/oka2015-applied-data-science/blob/main/colab/lecture03_tidyverse.ipynb

Thank you

See you on Friday for Lecture 4

Touseef Hameed | touseef.hameed@inn.no